COMPUTER SCIENCE DEPARTMENT

Computer Science – Curriculum Artificial Intelligence

Project Assignment – Database Systems

StageUp

Audiovisual Equipment Booking System

Database Design and Implementation

Student: *Valerio Flavio*

Academic Year 2025/2026

Contents

1 Conceptual Design **3**
- 1.1 Requirements . . . 3
- 1.2 Analysis . . . 4
  - 1.2.1 Reorganisation of Concepts . . . 4
  - 1.2.2 Glossary of Terms . . . 6
  - 1.2.3 Ambiguity Resolutions . . . 6
  - 1.2.4 Design Choice: Central Office Entity . . . 7
  - 1.2.5 Entity-Relationship Diagram . . . 8
  - 1.2.6 Entities and Attributes . . . 10
  - 1.2.7 Relationships . . . 12
  - 1.2.8 Business Rules (not expressible in the ER diagram) . . . 13

2 Logical Design **14**
- 2.1 Volume Table . . . 14
- 2.2 Access Table . . . 14
- 2.3 Redundancy Analysis . . . 17
  - 2.3.1 `Booking_Count` on Event_Location (for Operation 5) . . . 17
  - 2.3.2 `Installation_Number` on Team . . . 18
- 2.4 Relational Schema (logical model) . . . 18
- 2.5 UML Class Diagram . . . 19
- 2.6 Design Choices for Collections . . . 19
  - 2.6.1 Member – Standalone Table (not VARRAY) . . . 19
  - 2.6.2 Municipality – Nested Table inside Depot . . . 20

3 Implementation **21**
- 3.1 Object Types . . . 21
- 3.2 Tables . . . 23
- 3.3 Stored Procedures . . . 25
- 3.4 Triggers . . . 27

4 Physical Design **29**
- 4.1 Index Analysis . . . 29
  - 4.1.1 Operation 4 – View Teams at a Specific Event Location (20/day) 29
  - 4.1.2 Operation 5 – Ranked Locations (5/day) . . . 30
- 4.2 Performance Summary . . . 32

5 Web Interface **33**
- 5.1 Architecture . . . 33
- 5.2 Routes . . . 33

5.3 Backend Code 33
5.4 Frontend Screenshots 36

1 Conceptual Design

1.1 Requirements

"StageUp" – Audiovisual Equipment Booking System
The company "StageUp" has a central office and several operational depots located across the world, each covering a specific region, which consists of one or more municipalities, where they manage the rental and installation of audiovisual equipment for events. The central office does not handle physical setups. Both the central office and depots are characterized by name, address, city/province, and number of employees. Bookings, either recurring or one-time, are coordinated by the central office. Bookings can be placed by customers via phone, postal mail, email, or through the website. Besides one-time bookings, the company also handles seasonal contracts and special promotional rentals. Bookings are delivered and installed at customer event locations, which are characterized by address, house number, postal code, city, and province. Since each customer can have one or more event locations, each address is identified by a unique code within the customer's profile. Additionally, each event location has a setup time estimate and equipment capacity. Depots have one or more setup teams, each with a maximum number of members. Each team is characterized by an identification code, a name, the number of installations made, and some personal data of its team members. Each booking is characterized by type, date, duration, cost, the team that handled it, and the details of the customer who placed it. Customers, classified as individuals or companies, are identified by an alphanumeric code and their personal details.

Table 1: Requirements

1.2 Analysis

1.2.1 Reorganisation of Concepts

Depots and Central Office
The company operates through a single central office and several operational depots scattered across the world. The central office coordinates all the bookings but does not handle physical setups. The operational depots, on the other hand, manage the actual rental and installation of the audiovisual equipment. Both the central office and the depots are characterized by a name, address, city/province, and the number of employees. Additionally, each depot is assigned to cover a specific geographic region.

Table 2: Company Facilities Concepts

Team
Each depot has one or more setup teams. A team is identified by an identification code, a name, a maximum number of members, and the number of installations made. A team operates under exactly one depot.

Table 3: Team Concepts

Member
Each team is composed of members (specialized personnel). A member has personal data such as name, surname, phone, and email. Each member belongs to exactly one setup team.

Table 4: Member Concepts

Municipality
The specific region covered by a depot consists of one or more municipalities. Each municipality is covered by exactly one depot.

Table 5: Municipality Concepts

Customer
Customers place the bookings and can be classified as individuals or companies. They are identified by a unique alphanumeric code and their personal details. A customer can own one or more event locations.

Table 6: Customer Concepts

Event Location
An event location is a venue owned by a customer where the audiovisual equipment is delivered and installed. It is characterised by address, house number, postal code, city, province, a setup time estimate, and equipment capacity. Each location is uniquely identified within the specific customer's profile.

Table 7: Event Location Concepts

Booking
A booking represents the rental and installation of equipment requested by a customer. It is characterized by a type (recurring or one-time), date, duration, cost, booking method (phone, postal mail, email, website), and contract type (one-time, seasonal, promotional). Each booking is linked to the customer who placed it, the event location where it takes place, and the specific team that handled the physical setup.

Table 8: Booking Concepts

1.2.2 Glossary of Terms

Term	Description	Connections	Synonyms
Central Office	HQ coordinating bookings; no physical setup	Booking	HQ
Depot	Operational base; manages teams in a region	Team, Municipality	Branch
Region	Geographic area covered by a depot	Depot	Territory
Municipality	Town/area covered by exactly one depot	Depot	Town, Area
Team	Setup crew belonging to one depot	Booking, Member, Depot	Crew
Member	Employee of a team	Team	Staff, Technician
Customer	Individual or company placing bookings	Booking, Event_Location	Client
Event Location	Venue owned by a customer	Customer, Booking	Site, Venue
Booking	Audiovisual equipment installation reservation	Customer, Event_Location, Team	Reservation, Contract

Table 9: Glossary of Terms

1.2.3 Ambiguity Resolutions

1. **Central Office and bookings**: the central office *coordinates* bookings (business logic) but is not a direct participant in the ER relationships; the database records which team handles each booking, not which office coordinated it.

2. **Region**: not a standalone entity. It is an attribute of *Depot* representing the

geographic area covered. The region is further detailed by the municipalities it covers.

3. **Team–Depot cardinality**: "Depots have one or more setup teams" implies 1:N. Each team belongs to exactly one depot; a depot can have multiple teams.

4. **Booking type vs contract type**: "type" (recurring/one-time) describes the scheduling pattern; "contract type" (one-time/seasonal/promotional) describes the commercial arrangement. Both are attributes of *Booking*.

5. **Number of installations**: "number of installations made" is an attribute of Team. It is a *derived redundancy* (count of bookings handled) maintained automatically. This is a voluntary redundancy, consequence of a frequent operation.

1.2.4 Design Choice: Central Office Entity

The Central Office was intentionally excluded from being modeled as an independent entity within the conceptual schema to ensure greater efficiency and a cleaner design. Since it is a single, constant organizational unit for the entire system (essentially a "singleton"), translating it into a physical entity would result in the creation of a table containing only a single tuple. This would generate unnecessary overhead without providing any analytical advantage.

Furthermore, because the Central Office exclusively performs logistical coordination functions and does not actively participate in physical field installations, its role is considered an implicit premise of the business logic. Therefore, the fundamental relationships are established directly between Customers, Teams, and Bookings—namely, the entities that actively participate in the transactional flows of the database.

1.2.5 Entity-Relationship Diagram

SKELETON SCHEMA

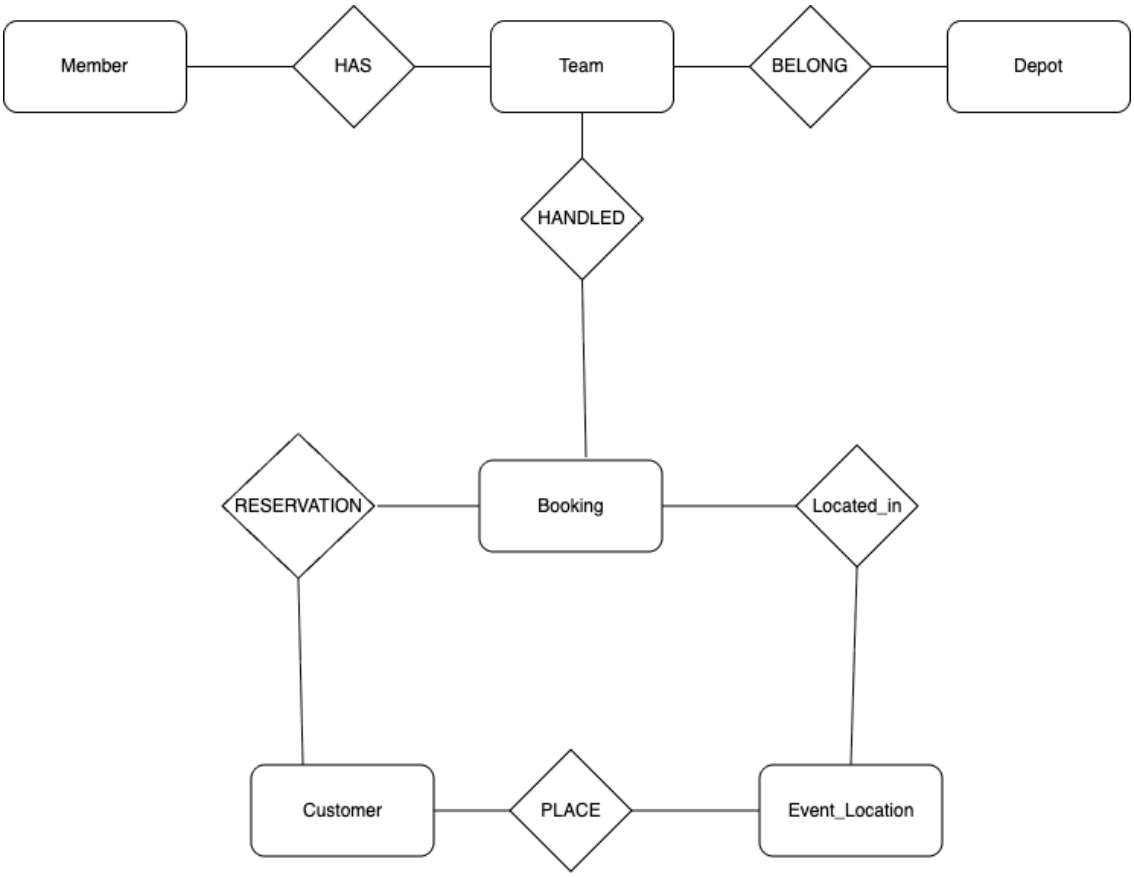


Figure 1: Skeleton Schema – Main entities and relationships (attributes and cardinalities omitted)

The skeleton schema shows the primary entities and the flow of the domain: **Customer** → **Booking** → **Team** → **Depot**, with **Event_Location** connecting Customer and Booking.

FINAL SCHEMA

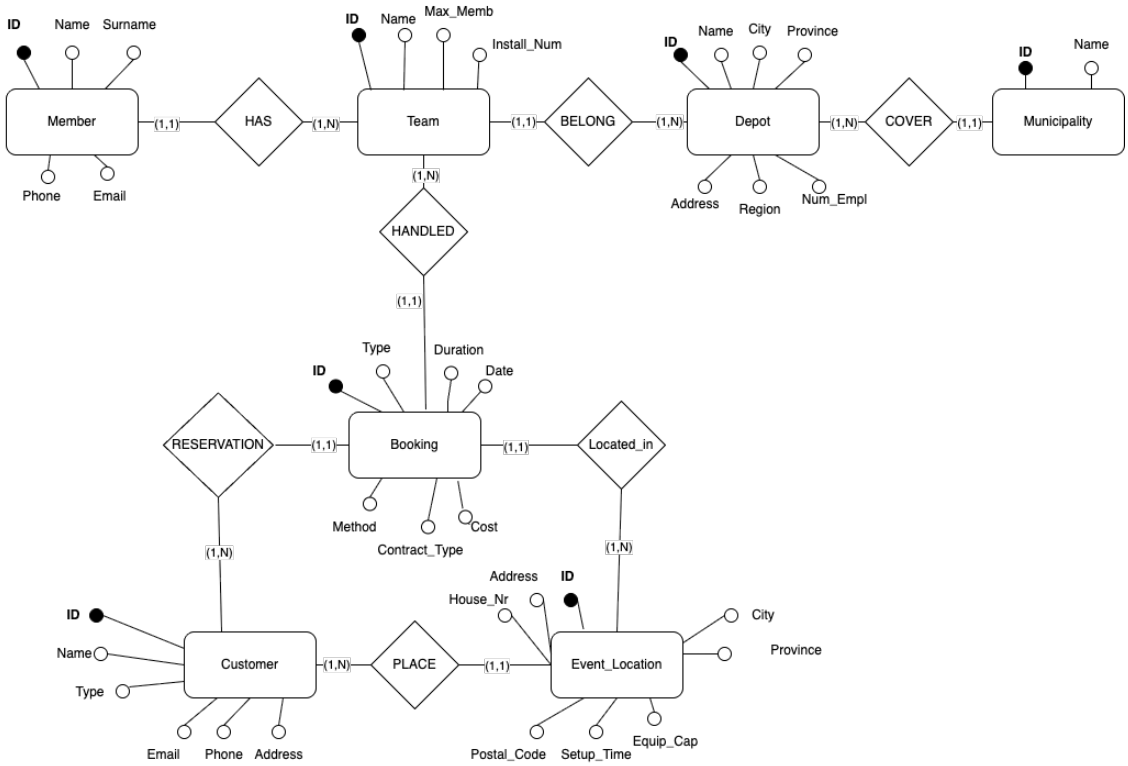


Figure 2: Final ER Schema with all entities, attributes, cardinalities and relationships.

1.2.6 Entities and Attributes

Entity	**Attributes**
Depot	ID (PK), Name, Address, City, Province, Number_Of_Employees
Municipality	ID (PK), Name (UNIQUE), Depot_ID (FK)
Team	Code (PK), Name, Max_Members, Installation_Number*, Depot_ID (FK)
Member	ID (PK), Name, Surname, Phone, Email, Team_Code (FK)
Customer	ID (PK), Name, Type $\in$ {Individual, Company}, Email, Phone, Address
Event_Location	ID (PK), Street, House_Number, Postal_Code, City, Province, Setup_Time_Estimate, Equipment_Capacity, Booking_Count*, Customer_ID (FK)
Booking	ID (PK), Type $\in$ {recurring, one-time}, Booking_Date, Duration, Cost, Booking_Method $\in$ {phone, email, website, postal_mail}, Contract_Type $\in$ {one-time, seasonal, promotional}, Team_Code (FK), Customer_ID (FK), Location_ID (FK)

Table 10: Entities and Attributes (* = derived redundant attribute)

1.2.7 Relationships

Name	Semantics	Cardinality
COVERS	A depot covers one or more municipalities; each municipality belongs to one depot	Depot (1,N) – Municipality (1,1)
BELONGS_TO	A team belongs to exactly one depot; a depot has one or more teams	Team (1,1) – Depot (1,N)
HAS	A team has one or more members; each member belongs to exactly one team	Team (1,N) – Member (1,1)
PLACE	A customer owns one or more event locations; each location belongs to one customer	Customer (1,N) – Event_Location (1,1)
RESERVATION	A customer makes one or more bookings; each booking is made by one customer	Customer (1,N) – Booking (1,1)
LOCATED_IN	An event location hosts one or more bookings; each booking is at one location	Event_Location (1,N) – Booking (1,1)
HANDLED	A booking is handled by exactly one team; a team handles one or more bookings	Booking (1,1) – Team (1,N)

Table 11: Relationships

1.2.8 Business Rules (not expressible in the ER diagram)

BR1 Booking.Duration > 0; Booking.Cost ≥ 0

BR2 Team.Max_Members > 0; Team.Installation_Number ≥ 0

BR3 Depot.Number_Of_Employees ≥ 0

BR4 Event_Location.Setup_Time_Estimate > 0; Equipment_Capacity ≥ 0

BR5 The number of members currently in a team must not exceed Team.Max_Members.

BR6 Team.Installation_Number is incremented automatically at each booking insert.

BR7 Event_Location.Booking_Count is incremented automatically at each booking insert.

2 Logical Design

2.1 Volume Table

The following volumes are given or derived from the problem statement. *Given*: 100 setup teams (max 10 members each), each depot operates in 10 municipalities, 1,000 event locations in total. Additional volumes are estimated (marked with †).

Concept	**Type**	**Volume**	**Why**
Depot	E	10	100 teams / 10 teams per depot
Municipality	E	100	10 per depot × 10 depots (given)
Team	E	100	Given
Member	E	700	Avg 7 members/team × 100 teams
Customer	E	500	1,000 locations / avg 2 per customer
Event_Location	E	1,000	Given
Booking	E	100,000	300 bookings/day × 365 days (given: 300/day) (*from op.2*)
COVERS	R	100	1:1 cardinality with municipality
BELONGS_TO	R	100	1:1 cardinality with depot
HAS	R	700	1:1 cardinality with member
PLACE	R	1,000	1:1 cardinality with event location
RESERVATION	R	100,000	1:1 cardinality with booking
LOCATED_IN	R	100,000	1:1 cardinality with booking
HANDLED	R	100,000	1:1 cardinality with booking

Table 12: Volume Table (yearly estimates; E = entity, R = relationship)

2.2 Access Table

Operation 1 – Register a new customer (10/day)

Concept	**Type**	**Accesses**	**R/W**
Customer	E	1	W

Table 13: Operation 1 – Total: $2 \times 10 = 20$ ops/day

Operation 2 – Record a new booking (300/day)

Concept	**Type**	**Accesses**	**R/W**
Customer	E	1	R (verify exists)
Event_Location	E	1	R (verify exists)
Team	E	1	R (verify exists)
Booking	E	1	W (create booking)
Booking	E	1	R (retrieve new booking id)
RESERVATION	R	1	W (link Customer → Booking)
LOCATED_IN	R	1	W (link Event_Location → Booking)
HANDLED	R	1	W (link Team → Booking)
Team	E	1	W (Installation_Number + 1)
Event_Location	E	1	W (Booking_Count + 1)

Table 14: Operation 2 – Total: $(1+1+1+2+1+2+2+2+2+2) \times 300 = 4{,}800$ ops/day

Please note: the accesses to update the redundant attributes (`Installation_Number` and `Booking_Count`) are included here, as they are part of the booking creation process, but they are managed by triggers and not directly by the operations.

Operation 3 – Register a new event location (50/day)

Concept	**Type**	**Accesses**	**R/W**
Customer	E	1	R (verify exists)
Event_Location	E	1	W (create location)
Event_Location	E	1	R (retrieve new location id)
PLACE	R	1	W (link Customer → Event_Location)

Table 15: Operation 3 – Total: $(1 + 2 + 1 + 2) \times 50 = 300$ ops/day

Operation 4 – View team(s) at a specific event location (20/day)

Concept	**Type**	**Accesses**	**R/W**
Event_Location	E	1,000	R (worst-case full table scan, N: 1,000)
LOCATED_IN	R	100	R (avg 100 bookings per location: 100,000/1,000)
Booking	E	100	R (retrieve each booking)
HANDLED	R	100	R (traverse to team for each booking)
Team	E	100	R (retrieve team info)

Table 16: Operation 4 – Total cost (assuming R=1, W=2): $(1{,}000 + 100 + 100 + 100 + 100) \times 20 = 28{,}000$ ops/day

Operation 5 – Event locations ranked by booking count descending (5/day)

Concept	**Type**	**Accesses**	**R/W**
Event_Location	E	1,000	R (scan all locations)

Table 17: Operation 5 (with `Booking_Count` redundancy) – Total: $1{,}000 \times 5 = 5{,}000$ ops/day

2.3 Redundancy Analysis

Two derived attributes are introduced as redundancies: `Installation_Number` on **Team** and `Booking_Count` on **Event_Location**.

2.3.1 `Booking_Count` on Event_Location (for Operation 5)

Without the redundancy, Op5 requires traversing all bookings via the `LOCATED_IN` relationship:

Concept	**Type**	**Accesses**	**R/W**
Event_Location	E	1,000	R
LOCATED_IN	R	100,000	R (traverse all bookings per location)
Booking	E	100,000	R (count all bookings)

Table 18: Op5 without `Booking_Count`

Cost Op5 without redundancy: $(1{,}000 + 100{,}000 + 100{,}000) \times 5 = 1{,}005{,}000$ ops/day. With the redundancy, Op2 must additionally update `Booking_Count`:

$$\Delta_{\text{Op2}} = 2 \times 300 = 600 \text{ ops/day extra}$$

Total cost comparison for the five operations in the two scenarios:

- **Total without redundancy:**
  $20 + 3{,}900 + 250 + 8{,}020 + 1{,}005{,}000 = \mathbf{1{,}017{,}190}$ ops/day
- **Total with redundancy:**
  $20 + 4{,}500 + 250 + 8{,}020 + 5{,}000 = \mathbf{17{,}790}$ ops/day

Conclusion: keeping `Booking_Count` reduces the total workload by a factor of ∼57. The redundancy is justified and maintained automatically by a trigger.

2.3.2 `Installation_Number` on Team

Without the redundancy, computing the installation count for a specific team requires finding all bookings handled by that team. Assuming a worst-case full table scan (no indexes on the foreign keys), the database must scan the entire Booking table:

$$\text{Reads per query} = \text{Volume(Booking)} = 100{,}000$$

The break-even (assuming this is queried q times per day):

$$q \times 100{,}000 > 300 \times 2 \quad \Rightarrow \quad q > 0.006$$

Since even a single query per day ($q = 1$) incurs 100,000 reads against a maintenance penalty of only 600 writes, keeping the redundancy is massively justified.

2.4 Relational Schema (logical model)

`Customer(`<u>`ID`</u>`, Name, Type, Email, Phone, Address)`
`Event_Location(`<u>`ID`</u>`, Street, House_Number, Postal_Code, City, Province, Setup_Time_Estimate, Equipment_Capacity, Booking_Count*, Customer_ID)`
`Booking(`<u>`ID`</u>`, Type, Booking_Date, Duration, Cost, Booking_Method, Contract_Type, Team_Code, Customer_ID, Location_ID)`
`Team(`<u>`Code`</u>`, Name, Max_Members, Installation_Number*, Depot_ID)`
`Member(`<u>`ID`</u>`, Name, Surname, Phone, Email, Team_Code)`
`Depot(`<u>`ID`</u>`, Name, Address, City, Province, Number_Of_Employees, Region, Municipalities: {(`<u>`ID`</u>`, Name)})`

Table 19: Relational Schema (* = redundant derived attribute; {} = nested collection embedded in parent)

2.5 UML Class Diagram

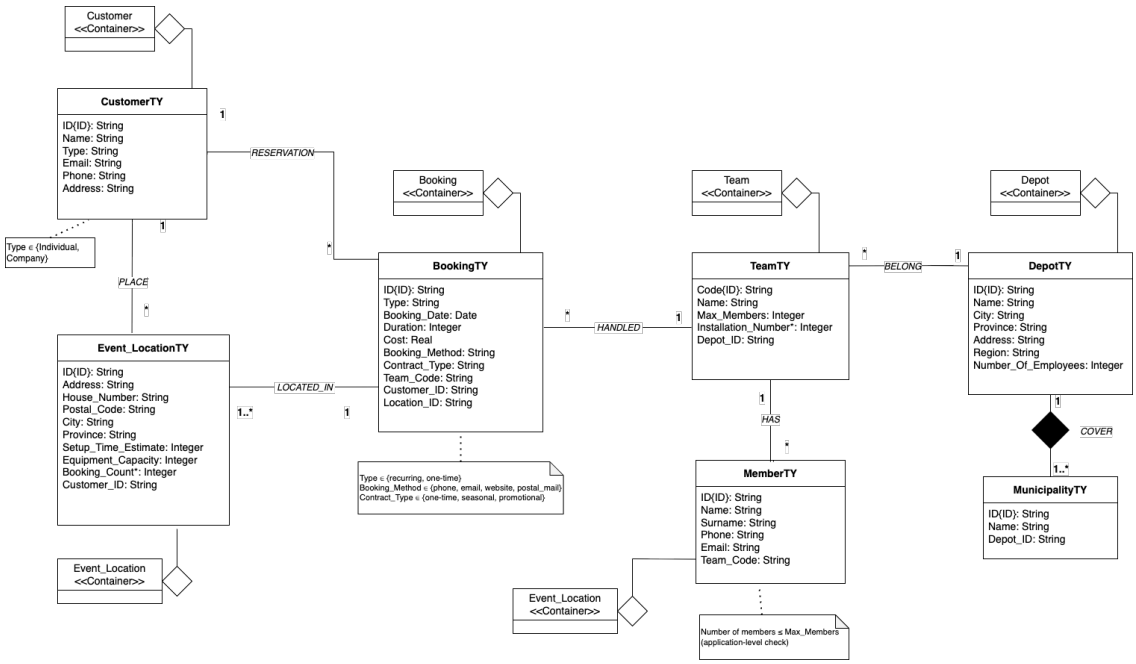


Figure 3: UML Class Diagram – Object-Relational Logical Schema, analysis phase (exported from `Draw/logic_model_uml.drawio`)

2.6 Design Choices for Collections

2.6.1 Member – Standalone Table (not VARRAY)

Members could in principle be modelled as a `VARRAY` of `MemberTY` inside `TeamTY`, with a fixed size of 10 reflecting the maximum stated in the assumed value for the volume table in the second exercise. This would make the capacity constraint self-enforcing at the storage level, eliminating the need for a dedicated trigger.

However, this approach was intentionally rejected for the following reason: `Team.Max_Members` is a *per-team attribute* that may vary across teams (e.g. a team could have a maximum of 3, 5, or 8 members rather than always 10). A `VARRAY(10)` would enforce only the global upper bound of the volume table, not the declared per-team limit. To correctly enforce `BR5` ($|\text{members}| \leq$ `Max_Members`), an application-level trigger is required regardless. Keeping `Member` as a standalone `OF TABLE` also preserves full relational queryability (e.g. finding all members of a team, or all teams a member belongs to) without resorting to the `TABLE()` unnesting operator.

2.6.2 Municipality – Nested Table inside Depot

`Municipality` is a *weak entity*: it has no meaningful existence outside its parent `Depot` (cardinality `(1,1)` on the municipality side of `COVERS`). This makes it a natural candidate for embedding as a collection inside `DepotTY`.

A `NESTED TABLE` of `MunicipalityTY` is chosen over a `VARRAY` because the number of municipalities per depot is unbounded in principle (the volume table gives 10 as an average, not a hard maximum), and nested tables support unbounded cardinality. The nested table is physically stored in a separate Oracle store table (`Municipality_Store`) but is logically part of `Depot`, reflecting the semantic dependency correctly.

This choice eliminates the standalone `Municipality` table and the `REF DepotTY` foreign reference that would otherwise appear in `MunicipalityTY`. None of the five defined operations access municipalities directly, so the restructuring has no impact on query performance or procedure logic.

3 Implementation

This section details the physical design of the system, implementing the structures defined in the logical schema. The database is developed on **Oracle Database** utilizing the **Object-Relational** model. Each entity is structured as an Oracle `OBJECT TYPE` and persisted within an `OF TABLE`. Inter-entity relationships are established using `REF` attributes to optimize object navigation.

3.1 Object Types

The object types representing the primary entities of the information system are defined first. For weak entities, such as Municipalities, a nested table approach is adopted, as they do not require independent references. Robust entities are defined as standard objects with their respective attributes. Redundant attributes (e.g., `Installation_Number` and `Booking_Count`), identified during the performance analysis to optimize read-heavy operations, are also included.

```
-- Municipality: element type for nested table (weak entity, no REF
    needed)
CREATE OR REPLACE TYPE MunicipalityTY AS OBJECT (
    ID   VARCHAR2(20),
    Name VARCHAR2(100)
);
-- Nested table type: unbounded collection of municipalities
CREATE OR REPLACE TYPE Municipality_NT AS TABLE OF MunicipalityTY;

-- Depot: embeds its municipalities as a nested table
CREATE OR REPLACE TYPE DepotTY AS OBJECT (
    ID                  VARCHAR2(20),
    Name                VARCHAR2(100),
    Address             VARCHAR2(200),
    City                VARCHAR2(100),
    Province            VARCHAR2(100),
    Number_Of_Employees INTEGER,
    Region              VARCHAR2(100),
    Municipalities      Municipality_NT
);

CREATE OR REPLACE TYPE CustomerTY AS OBJECT (
    ID      VARCHAR2(20),
    Name    VARCHAR2(100),
    Type    VARCHAR2(50),   -- 'Individual' | 'Company'
    Email   VARCHAR2(100),
    Phone   VARCHAR2(20),
    Address VARCHAR2(200)
);
```

```oracle_plsql
-- Team belongs to exactly one Depot (1:N BELONGS_TO)
CREATE OR REPLACE TYPE TeamTY AS OBJECT (
    Code                VARCHAR2(20),
    Name                VARCHAR2(100),
    Max_Members         INTEGER,
    Installation_Number INTEGER,   -- redundancy (count of bookings
        handled)
    Depot               REF DepotTY
);

CREATE OR REPLACE TYPE MemberTY AS OBJECT (
    ID      VARCHAR2(20),
    Name    VARCHAR2(100),
    Surname VARCHAR2(100),
    Phone   VARCHAR2(20),
    Email   VARCHAR2(100),
    Team    REF TeamTY
);

CREATE OR REPLACE TYPE Event_LocationTY AS OBJECT (
    ID                  VARCHAR2(20),
    Street              VARCHAR2(200),
    House_Number        VARCHAR2(20),
    Postal_Code         VARCHAR2(10),
    City                VARCHAR2(100),
    Province            VARCHAR2(100),
    Setup_Time_Estimate INTEGER,
    Equipment_Capacity  INTEGER,
    Booking_Count       INTEGER,   -- redundancy (for Op5)
    Customer            REF CustomerTY
);

CREATE OR REPLACE TYPE BookingTY AS OBJECT (
    ID             VARCHAR2(20),
    Type           VARCHAR2(50),   -- 'recurring' | 'one-time'
    Booking_Date   DATE,
    Duration       INTEGER,
    Cost           NUMBER,
    Booking_Method VARCHAR2(50),   -- 'phone'|'email'|'website'|'
       postal_mail'
    Contract_Type  VARCHAR2(50),   -- 'one-time'|'seasonal'|'
       promotional'
    Team           REF TeamTY,
    Customer       REF CustomerTY,
    Location       REF Event_LocationTY
);
```

3.2 Tables

Following the object types definition, the corresponding typed tables are created for physical data storage. Data integrity is enforced through `PRIMARY KEY` constraints, `NOT NULL` clauses, and specific `CHECK` constraints restricting values to predefined domains.

A critical design choice involves the referential integrity strategy, which prioritizes the preservation of historical audit trails over automated physical deletion. Destructive `ON DELETE CASCADE` policies are strictly avoided for robust entities. Specifically, the `Team` and `Depot` relationship utilizes the default `RESTRICT` behavior; this prevents the physical deletion of a depot or team if they are associated with historical bookings, enforcing a logical (soft) deletion approach at the application level to preserve service history.

For entities that can be safely detached, the `ON DELETE SET NULL` clause is applied. This is used in the `Member` table to ensure that if a team is conceptually dissolved, its employees are merely unassigned rather than deleted. Similarly, the `Event_Location` table utilizes `ON DELETE SET NULL` for its associated `Customer`, reflecting the business reality that a physical location persists and can be booked by other clients even if the original registering customer is removed. Conversely, the strict physical deletion of `Municipalities` tied to a `Depot` is handled natively and automatically by Oracle's nested table architecture without requiring explicit cascade commands.

```
-- Depot: municipalities embedded as nested table (weak entity)
CREATE TABLE Depot OF DepotTY (
    ID   PRIMARY KEY,
    Name NOT NULL,
    City NOT NULL,
    Address NOT NULL,
    Number_Of_Employees NOT NULL,
    CONSTRAINT chk_depot_employees CHECK (Number_Of_Employees >= 0)
) NESTED TABLE Municipalities STORE AS Municipality_Store;

-- Team: each team belongs to exactly one Depot (1:N BELONGS_TO)
CREATE TABLE Team OF TeamTY (
    Code PRIMARY KEY,
    Name NOT NULL,
    Max_Members NOT NULL,
    CONSTRAINT chk_max_members CHECK (Max_Members > 0),
    Installation_Number NOT NULL,
    CONSTRAINT chk_installation CHECK (Installation_Number >= 0),
    Depot NOT NULL REFERENCES Depot
);

CREATE TABLE Member OF MemberTY (
    ID      PRIMARY KEY,
```

```sql
    Name    NOT NULL,
    Surname NOT NULL,
    Team    REFERENCES Team ON DELETE SET NULL
);

CREATE TABLE Customer OF CustomerTY (
    ID   PRIMARY KEY,
    Name NOT NULL,
    CONSTRAINT chk_cust_type CHECK (Type IN ('Individual', 'Company
       ')),
    Type  NOT NULL,
    Email NOT NULL
);

CREATE TABLE Event_Location OF Event_LocationTY (
    ID     PRIMARY KEY,
    Street NOT NULL,
    City   NOT NULL,
    Setup_Time_Estimate NOT NULL,
    CONSTRAINT chk_setup_time CHECK (Setup_Time_Estimate > 0),
    Equipment_Capacity NOT NULL,
    CONSTRAINT chk_equip_cap CHECK (Equipment_Capacity >= 0),
    Booking_Count NOT NULL,
    CONSTRAINT chk_bk_count CHECK (Booking_Count >= 0),
    Customer REFERENCES Customer ON DELETE SET NULL
);

CREATE TABLE Booking OF BookingTY (
    ID PRIMARY KEY,
    CONSTRAINT chk_bk_type     CHECK (Type IN ('recurring','one-
       time')),
    CONSTRAINT chk_bk_method   CHECK (Booking_Method IN
        ('phone','email','website','postal_mail')),
    CONSTRAINT chk_ct_type     CHECK (Contract_Type IN
        ('one-time','seasonal','promotional')),
    CONSTRAINT chk_duration    CHECK (Duration > 0),
    CONSTRAINT chk_cost        CHECK (Cost >= 0),
    Type NOT NULL, Booking_Date NOT NULL,
    Duration NOT NULL, Cost NOT NULL,
    Booking_Method NOT NULL, Contract_Type NOT NULL,
    Team     NOT NULL REFERENCES Team,
    Customer NOT NULL REFERENCES Customer,
    Location NOT NULL REFERENCES Event_Location
);
```

3.3 Stored Procedures

To ensure business logic is handled efficiently within the database layer, stored procedures corresponding to the primary operations defined in the logical analysis are implemented. These procedures abstract the complexities of object-relational insertions and queries from the application layer.
The first procedure manages the registration of a new customer, encapsulating the physical insertion into the system and committing the transaction.

```
-- Procedure 1: Register a new customer (Operation 1)
CREATE OR REPLACE PROCEDURE proc_register_customer (
    p_id IN VARCHAR2, p_name IN VARCHAR2, p_type IN VARCHAR2,
    p_email IN VARCHAR2, p_phone IN VARCHAR2, p_address IN VARCHAR2
) AS
BEGIN
    INSERT INTO Customer VALUES (
        CustomerTY(p_id,p_name,p_type,p_email,p_phone,p_address));
    COMMIT;
END;
```

The second procedure orchestrates the creation of a booking. The existence of the requested Team, Customer, and Location is actively validated before generating the order to ensure referential safety.

```
-- Procedure 2: Record a new booking (Operation 2)
CREATE OR REPLACE PROCEDURE proc_add_booking (
    p_id IN VARCHAR2, p_type IN VARCHAR2, p_date IN DATE,
    p_duration IN INTEGER, p_cost IN NUMBER,
    p_method IN VARCHAR2, p_contract_type IN VARCHAR2,
    p_team_code IN VARCHAR2, p_customer_id IN VARCHAR2,
    p_location_id IN VARCHAR2
) AS
    v_ct NUMBER; v_cc NUMBER; v_cl NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_ct FROM Team          WHERE Code=
       p_team_code;
    SELECT COUNT(*) INTO v_cc FROM Customer       WHERE ID=
       p_customer_id;
    SELECT COUNT(*) INTO v_cl FROM Event_Location WHERE ID=
       p_location_id;
    IF v_ct=0 THEN RAISE_APPLICATION_ERROR(-20010,'Team not found')
       ; END IF;
    IF v_cc=0 THEN RAISE_APPLICATION_ERROR(-20011,'Customer not
       found'); END IF;
    IF v_cl=0 THEN RAISE_APPLICATION_ERROR(-20012,'Location not
       found'); END IF;
    INSERT INTO Booking VALUES (BookingTY(
```

```
        p_id, p_type, p_date, p_duration, p_cost, p_method,
           p_contract_type,
        (SELECT REF(t) FROM Team           t WHERE t.Code=
           p_team_code),
        (SELECT REF(c) FROM Customer       c WHERE c.ID=
           p_customer_id),
        (SELECT REF(l) FROM Event_Location l WHERE l.ID=
           p_location_id)));
    COMMIT;
END;
-- Note: triggers trg_update_installation_count and
   trg_update_booking_count
--       fire automatically after the INSERT.
```

When creating a new location, an associated customer is required. The third procedure handles this insertion while correctly initializing the redundant `Booking_Count` attribute to zero.

```
-- Procedure 3: Register a new event location (Operation 3)
CREATE OR REPLACE PROCEDURE proc_register_location (
    p_id IN VARCHAR2, p_street IN VARCHAR2, p_house_num IN VARCHAR2
       ,
    p_postal_code IN VARCHAR2, p_city IN VARCHAR2, p_province IN
       VARCHAR2,
    p_setup_time IN INTEGER, p_equip_cap IN INTEGER, p_customer_id
       IN VARCHAR2
) AS
    v_count NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_count FROM Customer WHERE ID=
       p_customer_id;
    IF v_count=0 THEN RAISE_APPLICATION_ERROR(-20020,'Customer not
       found'); END IF;
    INSERT INTO Event_Location VALUES (Event_LocationTY(
        p_id,p_street,p_house_num,p_postal_code,p_city,p_province,
        p_setup_time,p_equip_cap,
        0,  -- Booking_Count initialised to 0
        (SELECT REF(c) FROM Customer c WHERE c.ID=p_customer_id)));
    COMMIT;
END;
```

To satisfy reporting queries, the fourth procedure utilizes a `SYS_REFCURSOR` alongside the `DEREF` operator to efficiently navigate from a Booking to its assigned Team, returning a distinct list of teams for a specific location.

```
-- Procedure 4: View teams that handled a specific event location (
   Operation 4)
```

```oracle
CREATE OR REPLACE PROCEDURE proc_get_teams_at_location (
    p_location_id IN VARCHAR2, p_cursor OUT SYS_REFCURSOR
) AS
BEGIN
    OPEN p_cursor FOR
        SELECT DISTINCT
               DEREF(b.Team).Code AS team_code,
               DEREF(b.Team).Name AS team_name
          FROM Booking b
         WHERE b.Location = (SELECT REF(l) FROM Event_Location l
                             WHERE l.ID = p_location_id);
END;
```

Finally, the fifth procedure leverages the calculated `Booking_Count` redundancy. By pre-aggregating these values via triggers (detailed in the subsequent section), the query avoids costly real-time joins or groupings, returning a ranked list instantly.

```oracle
-- Procedure 5: Event locations ranked by booking count descending
   (Operation 5)
CREATE OR REPLACE PROCEDURE proc_get_locations_ranked (
    p_cursor OUT SYS_REFCURSOR
) AS
BEGIN
    OPEN p_cursor FOR
        SELECT l.ID, l.City, l.Street, l.Booking_Count
          FROM Event_Location l
         ORDER BY l.Booking_Count DESC;
END;
```

3.4 Triggers

To strictly enforce the business requirements (BR) derived during the analysis phase and dynamically maintain the consistency of redundant attributes, active database rules are implemented.

Before each member is inserted into a team, Trigger 1 reads the team's declared maximum capacity and the current count of its members. If the limit has been reached, the insertion is aborted, satisfying BR9 natively within the DBMS.

```oracle
-- Trigger 1: Enforce team member capacity (BR9)
CREATE OR REPLACE TRIGGER trg_member_capacity
BEFORE INSERT ON Member
FOR EACH ROW
DECLARE
    v_current_count NUMBER;
    v_max_members   NUMBER;
    v_team_code     VARCHAR2(20);
```

```
BEGIN
    SELECT t.Code, t.Max_Members INTO v_team_code, v_max_members
      FROM Team t WHERE REF(t) = :NEW.Team;
    SELECT COUNT(*) INTO v_current_count
      FROM Member m WHERE m.Team = :NEW.Team;
    IF v_current_count >= v_max_members THEN
        RAISE_APPLICATION_ERROR(-20040,
            'Error: Team ' || v_team_code ||
            ' has reached its max capacity (' || v_max_members || '
               ).');
    END IF;
END;
```

To maintain data consistency across performance-optimized tables, Triggers 2 and 3 operate silently upon every new booking creation. Firing `AFTER INSERT`, these triggers update the redundant counters (`Installation_Number` on the Team and `Booking_Count` on the Location) without any application-layer intervention, fulfilling BR10 and BR11.

```
-- Trigger 2: Increment Installation_Number after booking insert (
   BR10)
CREATE OR REPLACE TRIGGER trg_update_installation_count
AFTER INSERT ON Booking
FOR EACH ROW
BEGIN
    UPDATE Team t
       SET t.Installation_Number = t.Installation_Number + 1
     WHERE REF(t) = :NEW.Team;
END;
```

```
-- Trigger 3: Increment Booking_Count after booking insert (BR11 --
    Op5 redundancy)
CREATE OR REPLACE TRIGGER trg_update_booking_count
AFTER INSERT ON Booking
FOR EACH ROW
BEGIN
    UPDATE Event_Location l
       SET l.Booking_Count = l.Booking_Count + 1
     WHERE REF(l) = :NEW.Location;
END;
```

4 Physical Design

4.1 Index Analysis

Operations 1, 2, 3 access tables primarily by primary key; the implicit PK indexes are sufficient.

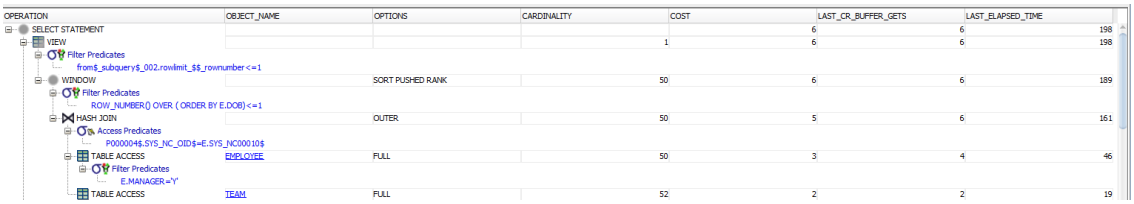
4.1.1 Operation 4 – View Teams at a Specific Event Location (20/day)

The query for Op4 scans `Booking` filtering by the `Location` REF column:

```
SELECT DISTINCT DEREF(b.Team).Code, DEREF(b.Team).Name
  FROM Booking b
 WHERE b.Location = (SELECT REF(l) FROM Event_Location l WHERE l.ID
     = :loc_id);
```

Without an index on `Booking.Location`, Oracle performs a **FULL TABLE SCAN** over all 100,000 bookings for each of the 20 daily executions.

Without index



OPERATION	OBJECT_NAME	OPTIONS	CARDINALITY	COST	LAST_CR_BUFFER_GETS	LAST_ELAPSED_TIME
SELECT STATEMENT				6	6	198
VIEW			1	6	6	198
Filter Predicates						
from$_subquery$_002.rowlimit_$$_rownumber<=1						
WINDOW		SORT PUSHED RANK	50	6	6	189
Filter Predicates						
ROW_NUMBER() OVER (ORDER BY E.DOB)<=1						
HASH JOIN		OUTER	50	5	6	161
Access Predicates						
P000004$.SYS_NC_OID$=E.SYS_NC00010$						
TABLE ACCESS	EMPLOYEE	FULL	50	3	4	46
Filter Predicates						
E.MANAGER='Y'						
TABLE ACCESS	TEAM	FULL	52	2	2	19

Figure 4: AutoTrace – Operation 4 without `idx_booking_location` (full table scan)

Index creation

```
CREATE INDEX idx_booking_location ON Booking(Location);
```

With index

OPERATION	OBJECT_NAME	OPTIONS	CARDINALITY	COST	LAST_CR_BUFFER_GETS	LAST_ELAPSED_TIME
SELECT STATEMENT				6	6	87
VIEW			1	6	6	87
Filter Predicates						
from$_subquery$_002.rowlimit_$$_rownumber<=1						
WINDOW		NOSORT STOPKEY	1	6	6	84
Filter Predicates						
ROW_NUMBER() OVER (ORDER BY E.DOB)<=1						
NESTED LOOPS		OUTER	1	6	6	77
TABLE ACCESS	EMPLOYEE	BY INDEX ROWID	50	5	4	31
Filter Predicates						
E.MANAGER='Y'						
INDEX	EMPLOYEE_DOB_INDEX	FULL SCAN	6	1	1	14
TABLE ACCESS	TEAM	BY INDEX ROWID	1	1	2	42
INDEX	SYS_C008566	UNIQUE SCAN	1	0	1	4
Access Predicates						
P000004$.SYS_NC_OID$=E.SYS_NC00010$						

Figure 5: AutoTrace – Operation 4 with `idx_booking_location` (index range scan)

With the index, Oracle limits the scan to the ∼100 bookings for the requested location.

4.1.2 Operation 5 – Ranked Locations (5/day)

Op5 reads all 1,000 `Event_Location` rows ordered by `Booking_Count DESC`. This is a full scan by design (we need all rows for ranking), so an index on `Booking_Count` is unlikely to be used by the optimiser for small-to-medium table sizes. The primary optimisation for Op5 is the `Booking_Count` **redundancy** (Section 2.3), which avoids a costly join with `Booking` (505,000 vs 5,000 ops/day).

Explain Plans

```
Descritto.


PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------------------------------------------------------------
Plan hash value: 3201819844

-------------------------------------------------------------------------------------
| Id  | Operation                | Name     | Rows  | Bytes | Cost (%CPU)| Time     |
-------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT         |          |     1 |    34 |     6  (17)| 00:00:01 |
|*  1 |  VIEW                    |          |     1 |    34 |     6  (17)| 00:00:01 |
|*  2 |   WINDOW SORT PUSHED RANK|          |    50 |  4300 |     6  (17)| 00:00:01 |
|*  3 |    HASH JOIN OUTER       |          |    50 |  4300 |     5   (0)| 00:00:01 |
|*  4 |     TABLE ACCESS FULL    | EMPLOYEE |    50 |  1100 |     3   (0)| 00:00:01 |
|   5 |     TABLE ACCESS FULL    | TEAM     |    52 |  3328 |     2   (0)| 00:00:01 |

PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------------------------------------------------------------
-------------------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   1 - filter("from$_subquery$_002"."rowlimit_$$_rownumber"<=1)
   2 - filter(ROW_NUMBER() OVER ( ORDER BY "E"."DOB")<=1)
   3 - access("P000004$"."SYS_NC_OID$"(+)="E"."SYS_NC00010$")
   4 - filter("E"."MANAGER"='Y')

Note

PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------------------------------------------------------------
-----
   - dynamic statistics used: dynamic sampling (level=2)

24 righe selezionate.
```

Figure 6: Explain Plan – Operation 4 with `idx_booking_location`

```
Descritto.


PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------------------------
Plan hash value: 3030065311

-----------------------------------------------------------------------------
| Id  | Operation          | Name     | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------
|   0 | SELECT STATEMENT   |          |   105 |  4200 |     3  (34)| 00:00:01 |
|   1 |  SORT ORDER BY     |          |   105 |  4200 |     3  (34)| 00:00:01 |
|   2 |   TABLE ACCESS FULL| FACILITY |   105 |  4200 |     2   (0)| 00:00:01 |
-----------------------------------------------------------------------------

Note

PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------------------------
-----
   - dynamic statistics used: dynamic sampling (level=2)

13 righe selezionate.
```

Figure 7: Explain Plan – Operation 5 (scan Event_Location with Booking_Count stored)

4.2 Performance Summary

Operation	Without optimisation	With optimisation
Op4	Full table scan on Booking (∼100,000 rows)	Index range scan (∼100 rows)
Op5	Join + count (∼101,000 reads, ×5/day)	Scan Event_Location only (1,000 reads, ×5/day)

Table 20: Performance summary for critical operations

5 Web Interface

The web interface is a **Flask** (Python) application that exposes the five operations defined in Exercise 2 via a browser-based HTML frontend, communicating with Oracle Database through the `oracledb` library.

5.1 Architecture

- **Backend**: `app.py` – Flask routes calling Oracle stored procedures via `callproc()` / `callfunc()`.
- **Frontend**: Jinja2 HTML templates with CSS.
- **Database**: Oracle XE on `localhost:1521/xe`.

5.2 Routes

URL	Method	Operation
`/`	GET	Homepage with links
`/register_customer`	GET/POST	Op1: Register new customer
`/add_booking`	GET/POST	Op2: Record new booking
`/register_location`	GET/POST	Op3: Register new event location
`/teams_at_location`	GET/POST	Op4: View teams at a location
`/ranked_locations`	GET	Op5: Locations ranked by booking count

Table 21: Flask routes and corresponding operations

5.3 Backend Code

```
# app.py - StageUp Audiovisual Equipment Booking System
from flask import Flask, render_template, request, redirect,
    url_for, flash
import oracledb, datetime

app = Flask(__name__)
app.secret_key = 'supersecretkey'
DB_USER='System'; DB_PASSWORD='password123'; DB_SID='localhost
    :1521/xe'

def get_db_connection():
```

```
    try: return oracledb.connect(user=DB_USER,password=DB_PASSWORD,
        dsn=DB_SID)
    except Exception as e: print("DB error:",e); return None

@app.route('/register_customer', methods=['GET','POST'])
def register_customer():
    if request.method=='POST':
        conn=get_db_connection()
        try:
            cur=conn.cursor()
            cur.callproc("proc_register_customer",[
                request.form['cust_id'], request.form['name'],
                request.form['cust_type'], request.form['email'],
                request.form['phone'],    request.form['address']])
            conn.commit(); flash("Customer registered","success")
        except oracledb.DatabaseError as e: flash(f"Error: {e}","
            danger")
        finally: cur.close(); conn.close()
        return redirect(url_for('index'))
    return render_template('register_customer.html')

@app.route('/add_booking', methods=['GET','POST'])
def add_booking():
    if request.method=='POST':
        d=datetime.datetime.strptime(request.form['booking_date','
            %Y-%m-%d').date()
        conn=get_db_connection()
        try:
            cur=conn.cursor()
            cur.callproc("proc_add_booking",[
                request.form['booking_id'], request.form['btype'],
                    d,
                int(request.form['duration']), float(request.form['
                    cost']),
                request.form['method'],       request.form['
                    contract_type'],
                request.form['team_code'],    request.form['
                    customer_id'],
                request.form['location_id']])
            conn.commit(); flash("Booking recorded","success")
        except Exception as e: flash(f"Error: {e}","danger")
        finally: cur.close(); conn.close()
        return redirect(url_for('index'))
    return render_template('add_booking.html')

@app.route('/register_location', methods=['GET','POST'])
def register_location():
    if request.method=='POST':
        conn=get_db_connection()
```

```
        try:
            cur=conn.cursor()
            cur.callproc("proc_register_location",[
                request.form['location_id'], request.form['street'
                    ],
                request.form['house_number'],request.form['
                    postal_code'],
                request.form['city'],        request.form['province
                    '],
                int(request.form['setup_time']),int(request.form['
                    equip_capacity']),
                request.form['customer_id']])
            conn.commit(); flash("Location registered","success")
        except Exception as e: flash(f"Error: {e}","danger")
        finally: cur.close(); conn.close()
        return redirect(url_for('index'))
    return render_template('register_location.html')

@app.route('/teams_at_location', methods=['GET','POST'])
def teams_at_location():
    teams=[]; location_id=None
    if request.method=='POST':
        location_id=request.form.get('location_id')
        conn=get_db_connection()
        try:
            cur=conn.cursor()
            out_cursor=cur.var(oracledb.CURSOR)
            cur.callproc("proc_get_teams_at_location",[location_id,
                out_cursor])
            teams=out_cursor.getvalue().fetchall()
        except Exception as e: flash(f"Error: {e}","danger")
        finally: cur.close(); conn.close()
    return render_template('teams_at_location.html',
                           teams=teams, location_id=location_id)

@app.route('/ranked_locations')
def ranked_locations():
    locations=[]; conn=get_db_connection()
    try:
        cur=conn.cursor()
        out_cursor=cur.var(oracledb.CURSOR)
        cur.callproc("proc_get_locations_ranked",[out_cursor])
        locations=out_cursor.getvalue().fetchall()
    except Exception as e: flash(f"Error: {e}","danger")
    finally: cur.close(); conn.close()
    return render_template('ranked_locations.html',locations=
        locations)

if __name__=='__main__': app.run(debug=True)
```

5.4 Frontend Screenshots

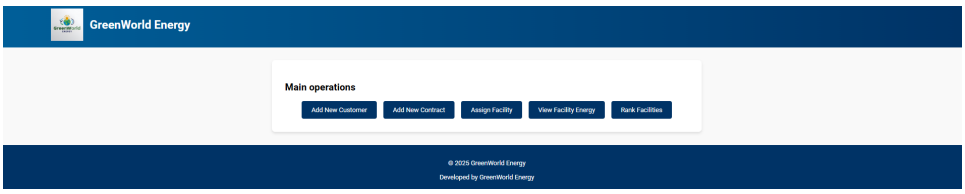


Figure 8: Homepage

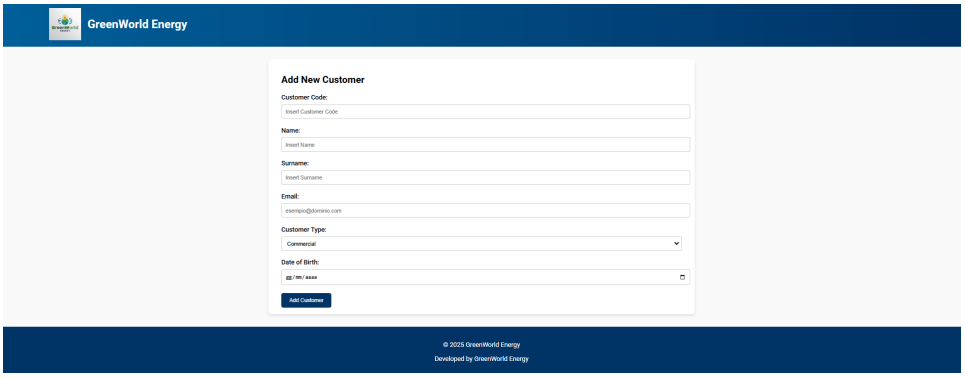


Figure 9: Operation 1 – Register a new customer

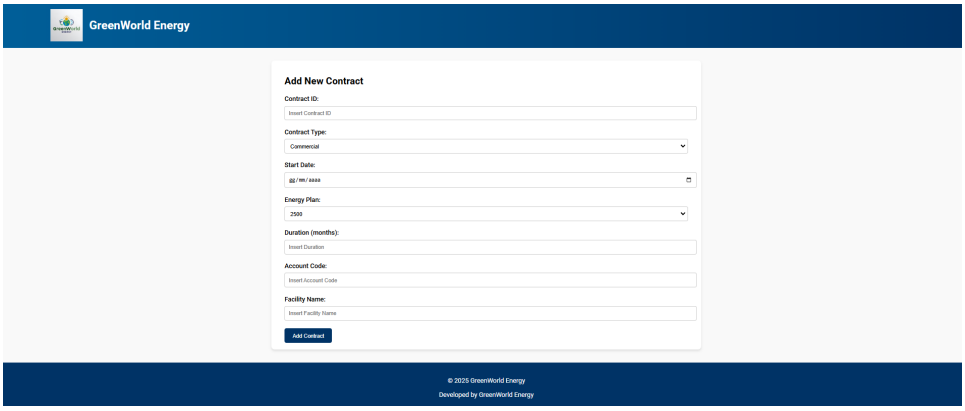


Figure 10: Operation 2 – Record a new booking

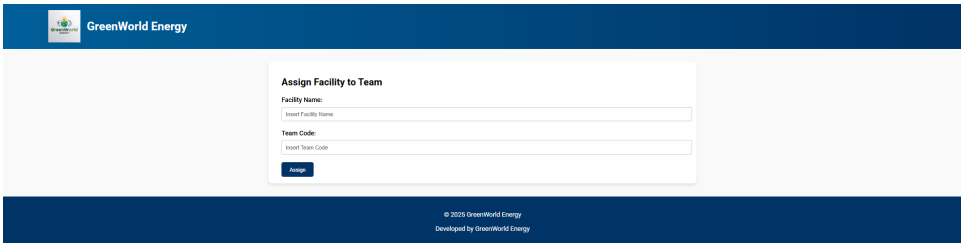


Figure 11: Operation 3 – Register a new event location

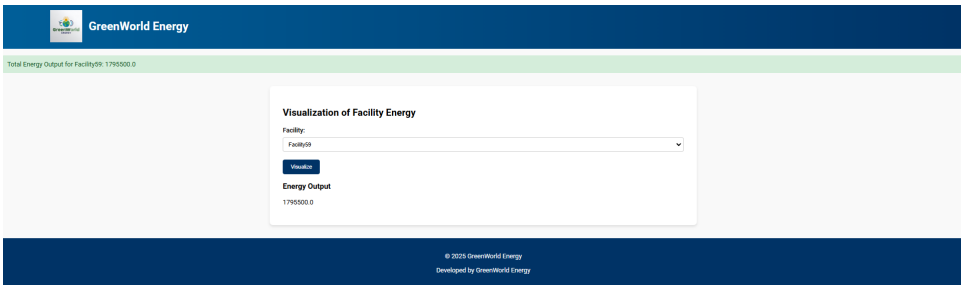
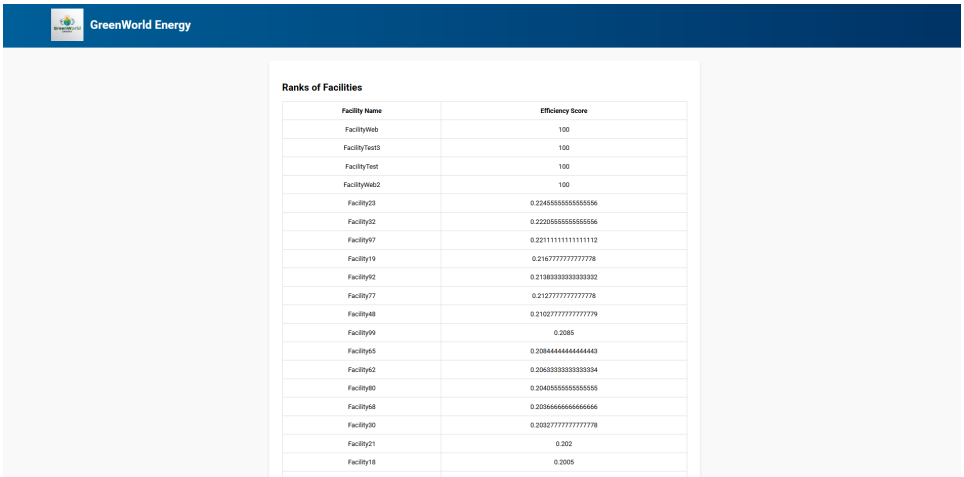


Figure 12: Operation 4 – View teams that handled setups at an event location



Facility Name	Efficiency Score
FacilityWeb	100
FacilityTest3	100
FacilityTest	100
FacilityWeb2	100
Facility23	0.2245555555555556
Facility32	0.2220555555555556
Facility97	0.2211111111111112
Facility19	0.216777777777778
Facility92	0.2138333333333332
Facility77	0.212777777777778
Facility48	0.2102777777777779
Facility99	0.2085
Facility65	0.2084444444444443
Facility62	0.2063333333333334
Facility80	0.2040555555555555
Facility68	0.2036666666666666
Facility30	0.2032777777777778
Facility21	0.202
Facility18	0.2005

Figure 13: Operation 5 – Event locations ranked by booking count (descending)